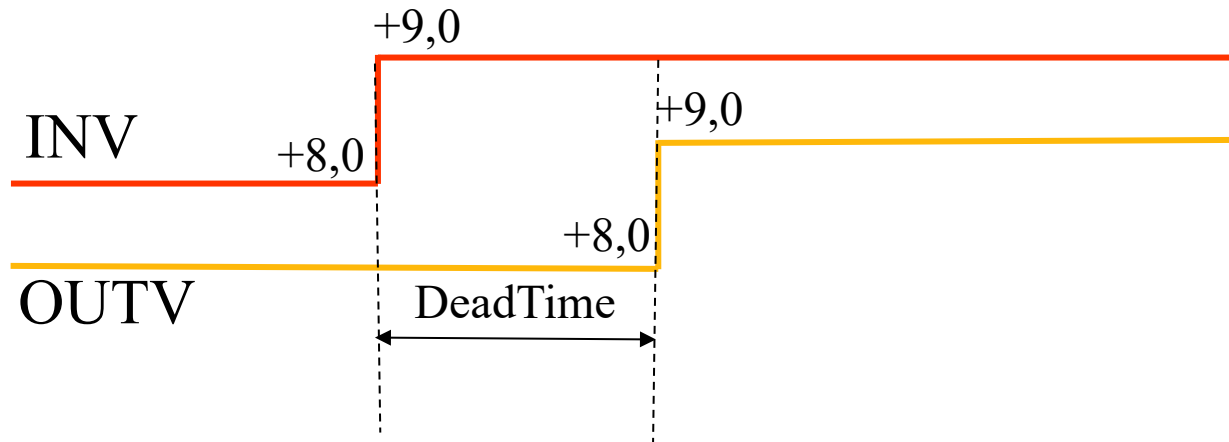
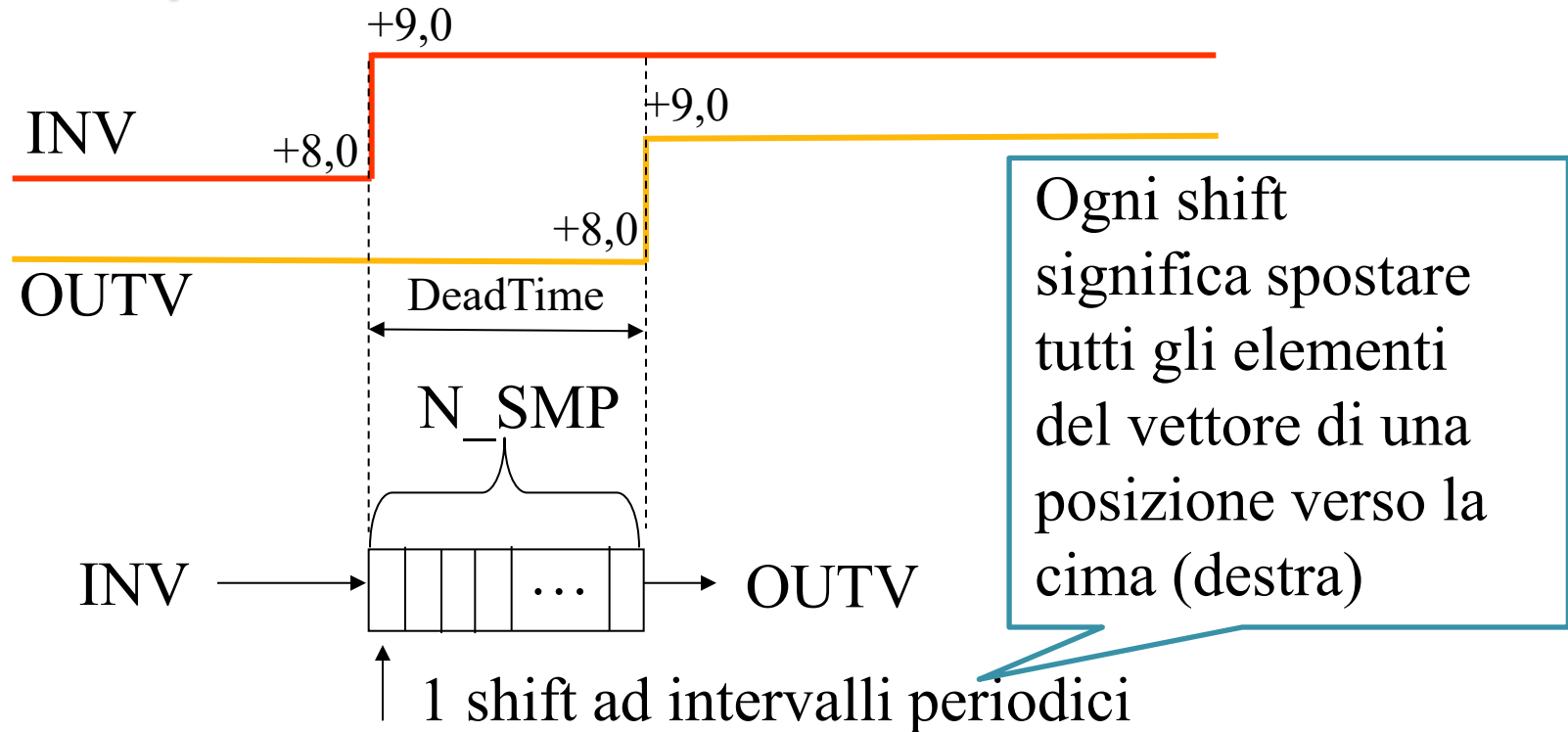


Esempio Task Ciclico

- ❖ Supponiamo si voglia realizzare un programma che crei un ritardo su un segnale (analogico) di ingresso
- ❖ Nella figura:
 - INV = segnale in ingresso
 - OUTV = segnale in uscita pari a INV, ma ritardato di un intervallo temporale (DeadTime).

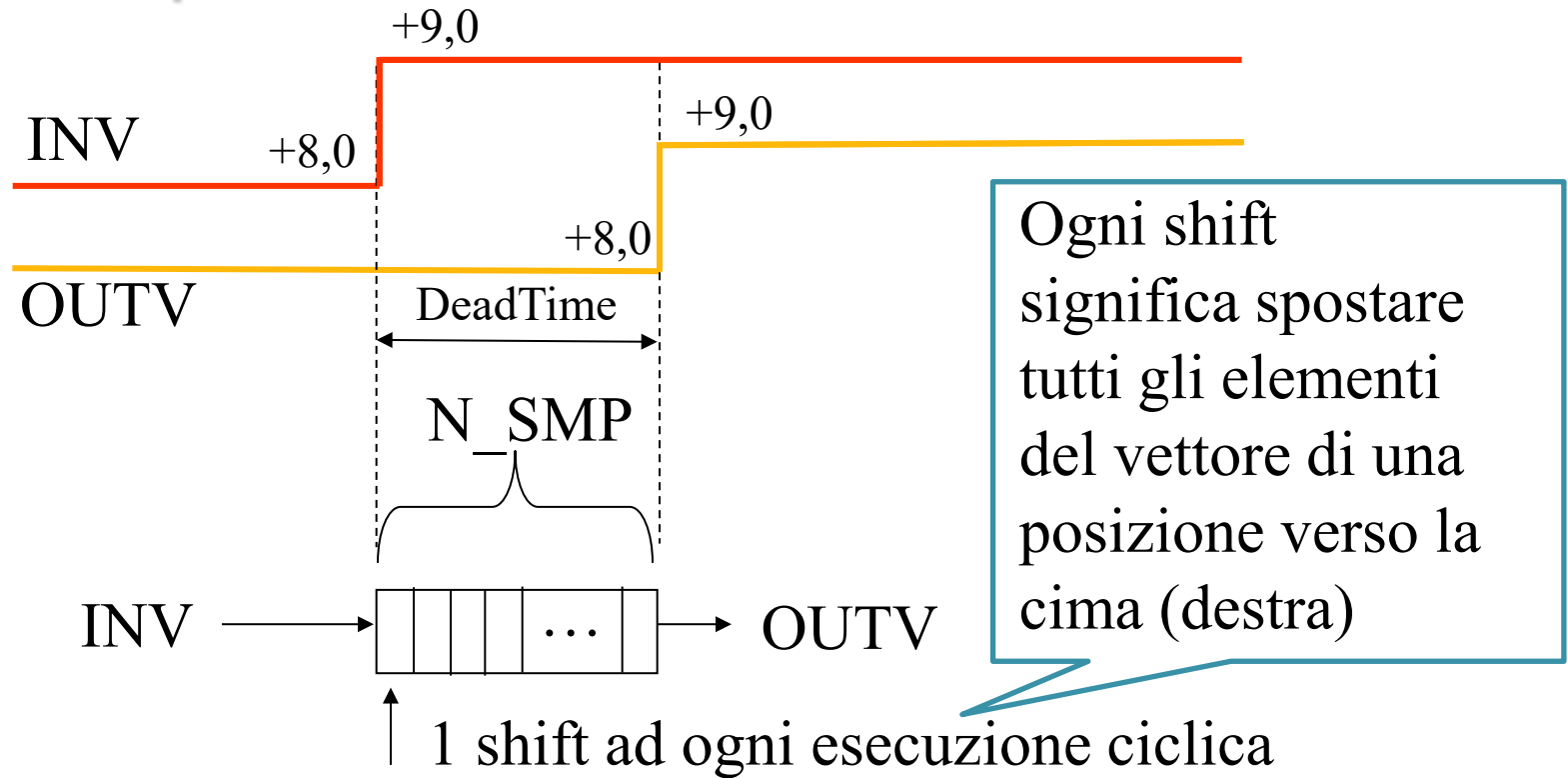


Esempio Task Ciclico



- ❖ Viene utilizzato un vettore di un numero massimo di elementi (memorizza una sequenza di segnali di ingresso INV)
- ❖ **Viene effettuato uno shift ad intervalli periodici**
- ❖ N_SMP rappresenta il numero di shift dopo cui il segnale in ingresso viene prodotto in uscita
- ❖ **$DeadTime = N_SMP * Intervallo$**
- ❖ **Soluzione: il programma di shift viene controllato da un task ciclico**

Esempio Task Ciclico

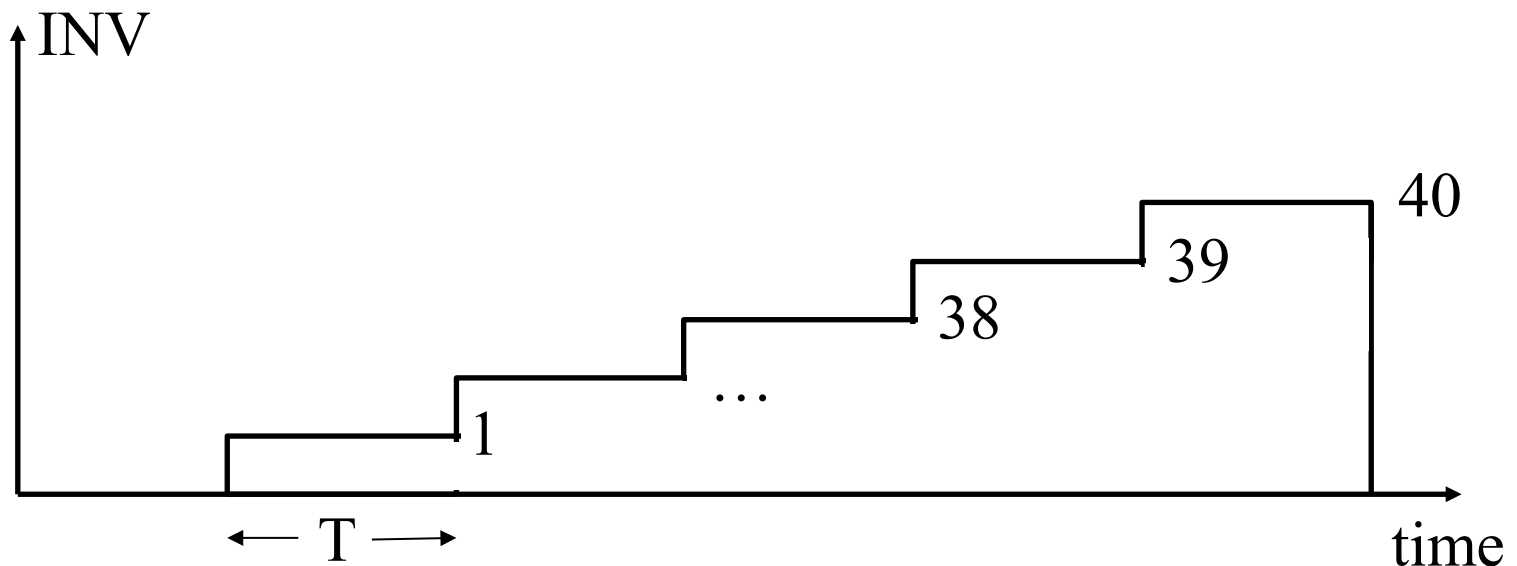


Esempio:

- ❖ Il segnale della variabile di ingresso "INV" deve essere ritardato di un 1s
- ❖ Supponendo che l'intervallo ciclico del task sia 100 ms.
- ❖ "N_SMP" deve essere posto a 10
- ❖ **$\text{DeadTime} = \text{N_SMP} * \text{Intervallo ciclico} = 10 * 100 \text{ ms} = 1 \text{ s}$**

Esempio Task Ciclico

- ❖ Non potendo disporre di un segnale analogico reale, lo simuliamo
- ❖ Supporremo che l'ingresso analogico INV sia un segnale a gradino che viene incrementato in modo unitario nell'intervallo $[0,40]$,
- ❖ T =si suppone che sia lo stesso **Intervallo ciclico**



Esempio Task Ciclico

Soluzione Adottata:

- ❖ 2 Task ciclici con lo stesso periodo T
- ❖ Il primo task ciclico contiene il programma che genera il segnale a gradino
- ❖ Il secondo task ciclico contiene il programma che esegue lo shift come spiegato prima
- ❖ Avendo lo stesso periodo, si preferisce differenziare l'esecuzione dei due task, tramite **offset**
- ❖ Si eseguirà prima il task che produce il segnale e dopo pochi ms, si esegue il task che produce lo shift

Esempio Task Ciclico

segnale di uscita

segnale di ingresso

valore effettivo di
N_SMP

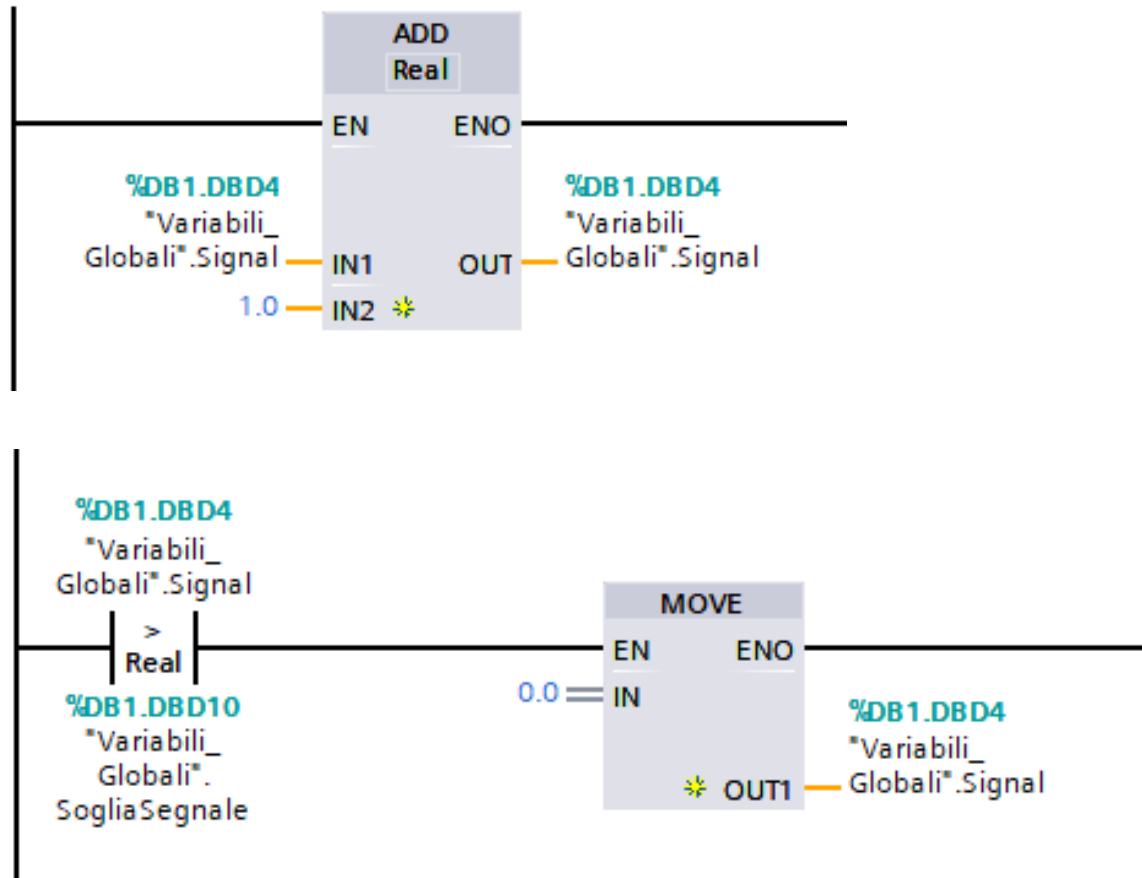
valore soglia per il
segnale a gradino

Variabili_Globali						
	Nome	Tipo di dati	Offset	Valore di avvio	Ritenzione	
1	Static				☐	
2	D_Signal	Real	0.0	0.0	☒	
3	Signal	Real	4.0	0.0	☒	
4	SampleNumber	Int	8.0	5	☒	
5	SogliaSegnale	Real	10.0	4.0e+1	☒	

- ❖ Supporremo che sia possibile memorizzare Max 20 ritardi (N_SMP può assumere il valore 20 al massimo)
- ❖ Supporremo che il numero effettivo di ritardi possa essere deciso a run-time (al massimo N_SMP)
- ❖ Supporremo anche di parametrizzare il valore di soglia del segnale gradino (per default è 40, ma può essere cambiato)
- ❖ Definiamo uno struct, con i campi:

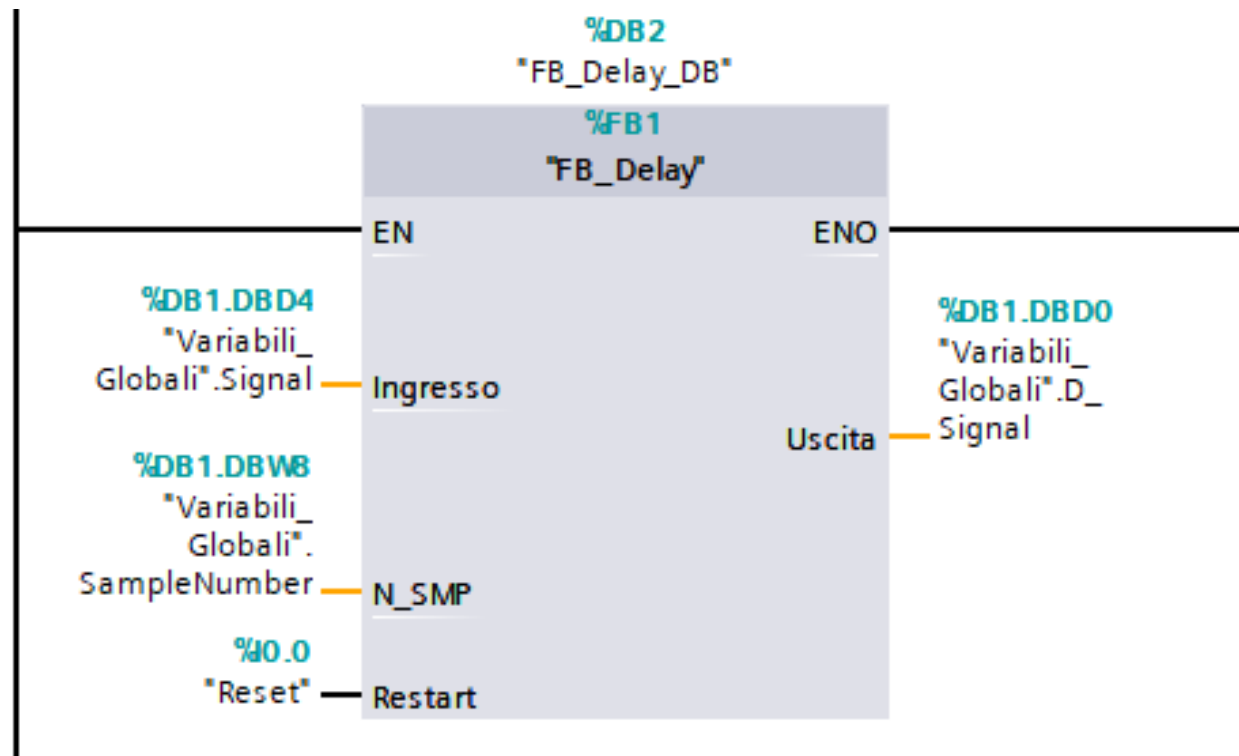
Esempio Task Ciclico

- ❖ Programma che produce il segnale a gradino ed è controllato dal primo task periodico di periodo T:



Esempio Task Ciclico

- ❖ Definiamo adesso un FB che realizza l'algoritmo di shift
- ❖ Questo FB verrà controllato da un secondo task periodico con lo stesso periodo T, ma con un offset di pochi ms



Variabili del Function Block **FB_Delay**:

- ❖ Ingresso: tipo REAL, è il segnale di ingresso
- ❖ N_SMP: tipo INT, è il valore attuale di N_SMP
- ❖ Restart: tipo BOOL è il reset

Variabile di uscita:

- ❖ Uscita: tipo REAL, è il segnale di uscita

Variabile statica:

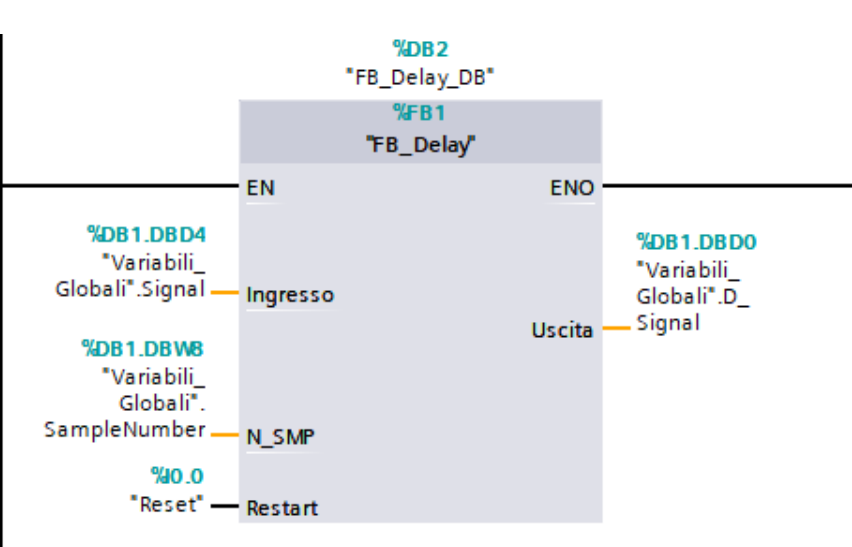
- ❖ STORE: memoria dati ARRAY di REAL

➤ 20 elementi al massimo

- ❖ indice_1, indice_2: tipo INT (indici vettore)
- ❖ sup: INT (indice vettore)

Nota Bene:

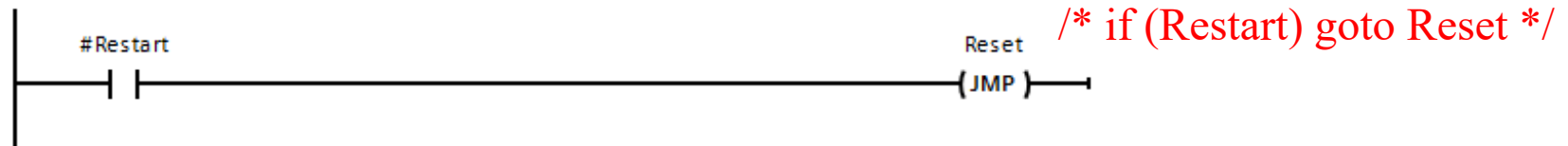
- ❖ Gli Input possono non essere a ritenzione perché sono connessi a variabili a ritenzione e ad un ingresso (%I0.0, che viene aggiornato ad ogni ciclo)



FB_Delay				
	Nome	Tipo di dati	Valore di default	Ritenzione
1	Input			
2	Ingresso	Real	0.0	Non a ritenzione
3	N_SMP	Int	0	Non a ritenzione
4	Restart	Bool	false	Non a ritenzione
5	Output			
6	Uscita	Real	0.0	Non a ritenzione
7	InOut			
8	<Inserisci>			
9	Static			
10	STORE	Array[0..19] of Real		A ritenzione
11	indice_1	Int	0	A ritenzione
12	indice_2	Int	0	A ritenzione
13	sup	Int	0	A ritenzione
14	Temp			
15	<Inserisci>			
16	Constant			
17	<Inserisci>			

Esempio Task Ciclico

❖ Porzione di Codice per il Reset (attivato se Restart==1):

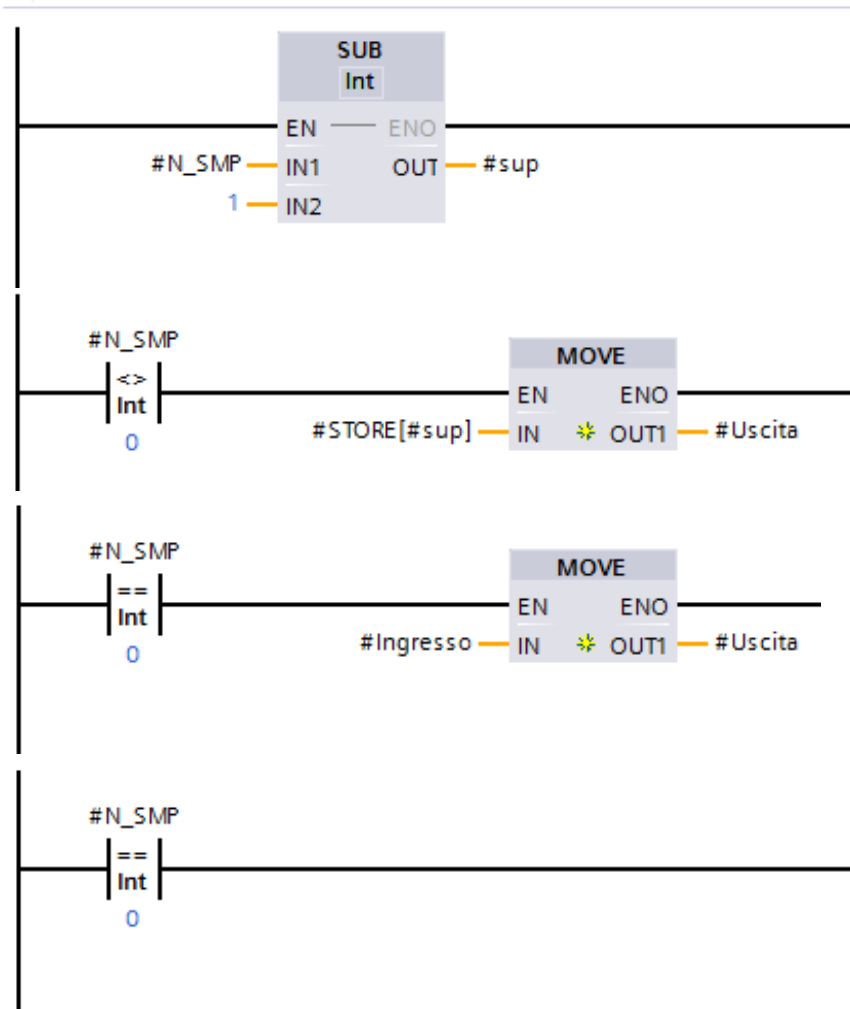


`/* blocco di istruzioni alla fine del codice */`



Esempio Task Ciclico

❖ Porzione di Codice per la "produzione" dell'uscita:



$sup = N_SMP - 1$

if ($N_SMP \neq 0$) Uscita = STORE[sup]

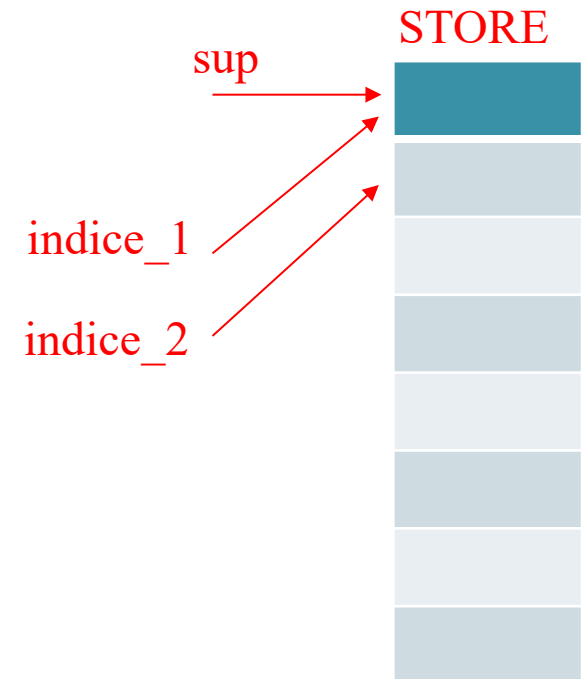
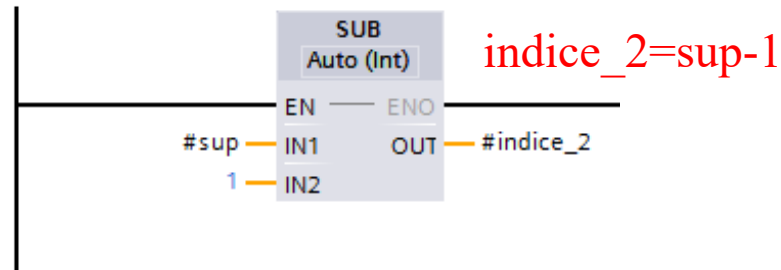
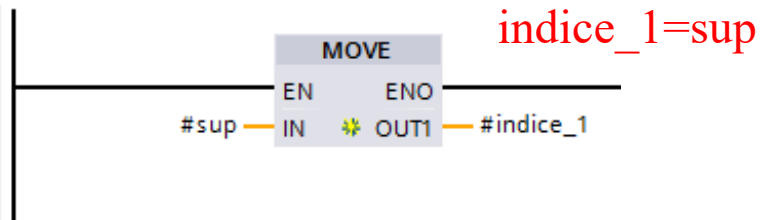
if ($N_SMP == 0$) Uscita = Ingresso

if ($N_SMP == 0$) va alla fine del codice
Così salta tutto il codice del lucido
successivo

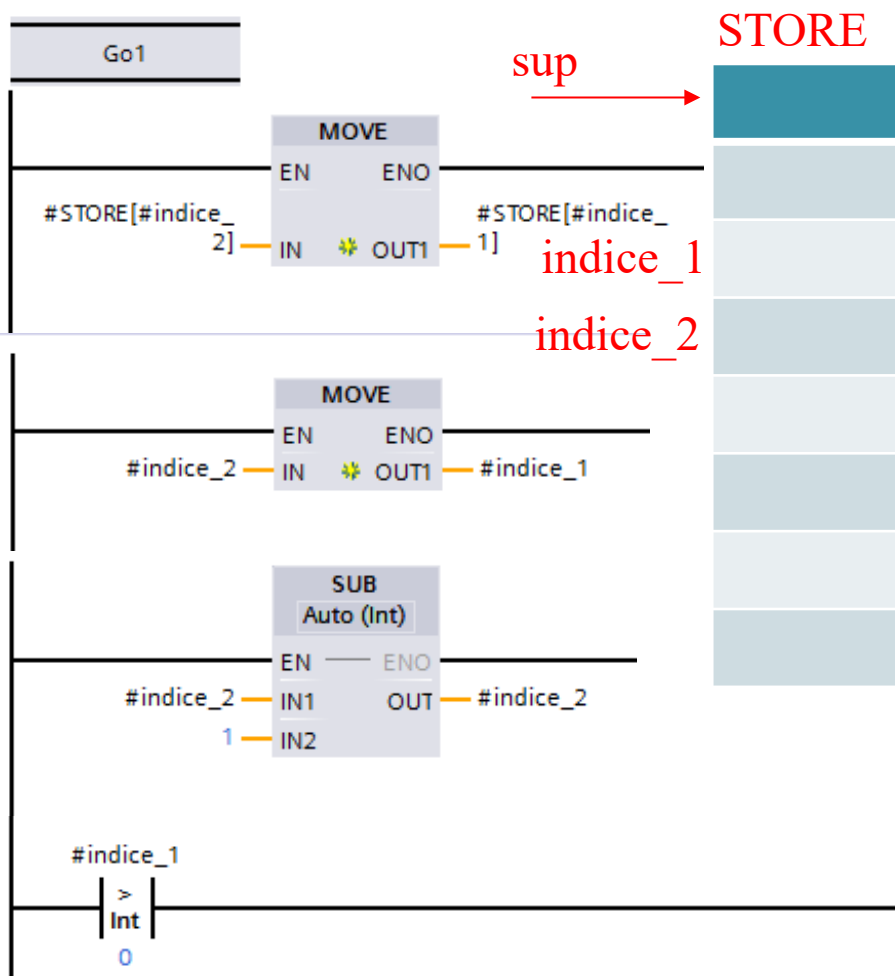
Esempio Task Ciclico

❖ Porzione di Codice per lo "shift" dell'ingresso :

- inizializzazione delle variabili indice_1 e indice_2



❖ Porzione di Codice per lo "shift" dell'ingresso :



STORE

sup

indice_1

indice_2

```
do{  
    STORE[indice_1]=STORE[indice_2];  
    indice_1=indice_2;  
    indice_2=indice_2-1;  
} while (indice_1>0);
```

/*esegue l'immissione dell'ingresso nel primo elemento del vettore: STORE[0]=Ingresso; */

